

AI Security 5.3. Sensitive Information Disclosure

Task 1 - Introduction

Large Language Models generate responses based on patterns learned during training and on data retrieved at runtime. Unlike traditional databases, they do not enforce strict row-level access rules by default. This creates a new class of confidentiality risk: sensitive information disclosure. In LLM systems, disclosure does not usually happen because a model "decides" to reveal a secret. It happens because sensitive data was allowed to enter the system's context window.

Task 2 - OWASP LLM02

Exposures from sensitive information disclosures are categorised under OWASP LLM02. They focus on protecting private data, proprietary logic, and confidential documents from being exposed through AI outputs.



LLM02 - Disclosure happens when authorization is not enforced before retrieval.

Disclosure Is Different From Poisoning and Prompt Injection

- Poisoning manipulates what the system learns.
- Prompt injection manipulates instructions at runtime.
- Sensitive Information Disclosure exposes data that already exists in the system.

The attacker does not need to modify the model. They only need to trigger retrieval or observe architectural weaknesses.

How Data Leakage Happens in Real Systems: Modern LLM applications often use Retrieval-Augmented Generation (RAG). Instead of relying only on training data, the system retrieves documents from a vector database and injects them into the model's context before generating a response. This creates multiple potential leakage points:

- Over-broad similarity search retrieves unrelated confidential documents
- Shared vector databases across departments or tenants
- Large context windows combining multiple document chunks
- Debug logging stores full prompts and retrieved content
- Weak or missing metadata filtering

If unauthorised data is retrieved, the exposure risk already exists even if the model only partially reveals it. The issue is architectural, not conversational.

Scenario: Exposure Without Breaking Anything: *A cloud infrastructure provider launches an AI-powered support assistant for customers. The assistant helps users troubleshoot deployment errors, billing issues, and configuration problems. The system uses RAG. It retrieves information from:*

- *Public documentation*
- *Internal troubleshooting playbooks*
- *Archived support tickets*
- *Incident postmortems*

To improve answer quality, the engineering team decides to embed past support tickets into the vector database. These tickets often contain:

- *Customer account IDs*
- *Deployment architecture diagrams*
- *API keys accidentally pasted during troubleshooting*
- *Root cause summaries*

The team assumes this data is safe because the tickets are internal, the model is stateless, and the system prompt instructs the model not to reveal secrets.

A customer asks: "Why does my Kubernetes deployment keep failing when autoscaling triggers?"

The retrieval system performs a similarity search across all embedded tickets. One archived ticket from another customer is highly similar. It contains a detailed breakdown of the autoscaling misconfiguration along with the customer's account ID and the internal cluster naming scheme. The retrieved results include that ticket. The model generates a helpful explanation. While answering, it includes a short example configuration snippet copied from the archived ticket. The snippet contains: `cluster-prod-eu-west-42`. That cluster belongs to another customer. The assistant just exposed part of another tenant's infrastructure naming convention.

No database permissions were bypassed. No authentication failed. The model did not memorise anything. The retrieval system pulled semantically similar data without enforcing tenant isolation. Weeks later, during a compliance audit, security discovers that full augmented prompts, including entire ticket contents, were logged in the observability platform. The logging system had broader access permissions than the ticketing system itself. The failure was not in the generation. It was in the retrieval and logging architecture. The system is optimised for relevance. It did not enforce isolation.

Why Sensitive Information Disclosure Is Dangerous: Disclosure often appears to be normal system behaviour. The AI still answers confidently. The system does not crash. No error is thrown. Confidentiality failures in LLM systems are subtle because:

- Vector stores retain embedded documents until they are explicitly removed
- Embeddings approximate semantic relationships by positioning similar text close together in a geometric vector space

- Logging systems often have broader access than source databases
- Access control may be implemented in prompts instead of being enforced at the database layer

If retrieval is not deterministic and access-controlled, sensitive information can silently move across trust boundaries.

Answer the questions below

| What OWASP category covers sensitive data exposure in LLM systems?

LLM02

Task 3 - Common Disclosure Scenarios

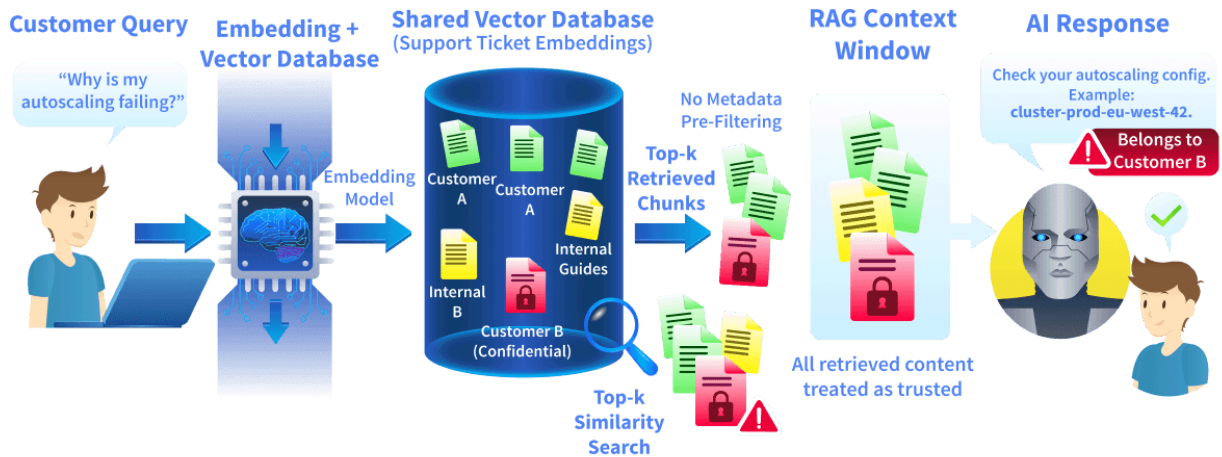
Sensitive information disclosure rarely begins with a dramatic exploit. It begins with normal system behaviour:

- The retrieval layer queries documents
- A prompt is constructed
- An answer is generated

The problem is not that the model is malicious. The problem is that retrieval and logging layers treat data as trusted before verifying authorisation. In this task, we examine how disclosure occurs in practice, not through hacking, but through architectural assumptions.

Scenario: The Multi-Tenant Support Platform

The Multi-Tenant Support Platform - When Similarity Breaks Isolation



No hack. No exploit. The leak happened at retrieval.

A technology company deploys an internal AI assistant to help engineers troubleshoot production incidents. The assistant uses RAG. When a user submits a query, the system:

1. Retrieves the top-k most relevant internal documents
2. Constructs an augmented prompt (User Query + Retrieved Chunks + System Prompt)
3. Sends the prompt to the LLM
4. Logs the full request for debugging and quality monitoring

To diagnose hallucinations, the engineering team enables verbose logging in production. Every request now records:

- The user's query
- All retrieved document chunks
- The full augmented prompt
- The model's final response

One evening, a senior engineer asks: "Summarise last quarter's authentication failure trends." The retrieval system correctly pulls: public reliability metrics, an

internal security incident report, and a confidential breach analysis document.

The engineer is authorised to see this information. The system generates a useful summary. Nothing appears wrong. Later, during a routine audit, security discovers something unexpected. All augmented prompts are stored in plaintext in the centralised logging platform. The logging system is used by DevOps, support analysts, and external contractors with monitoring access. These users do not have permission to access security incident reports directly. But they now have access to the full prompt including the confidential breach analysis text.

No database permissions were bypassed. No retrieval logic failed - no unauthorised user queried the AI. The exposure occurred because logging inherited broader access than the source system. The AI system correctly enforced access at retrieval. The logging architecture silently broke that boundary.

What Actually Happened: In a RAG system, the augmented prompt contains:

- User input
- Retrieved sensitive documents (determined by the top-k retrieval setting)
- System-level instructions

If that prompt is logged without redaction, the log becomes a secondary data store. Often, logging systems have:

- Broader access controls
- Longer retention policies
- Less granular permissions

The result is a data boundary failure through observability. The model did not leak the secret in its answer. The infrastructure leaked it in its logs.

Case Study 1: EchoLeak (CVE2025-32711): *The EchoLeak vulnerability(opens in new tab)* demonstrated how disclosure can occur through retrieval trust assumptions in production RAG systems. In this case, a malicious email was sent to a target user. The email contained hidden instructions embedded inside HTML content. These instructions were not visible to the user but were parsed and indexed by the system. When the AI assistant later processed a benign request such as "Summarise my unread emails," the retrieval layer selected the malicious email because its embedding was geometrically close to the query vector in high-

dimensional space. The retrieved email content entered the context window alongside trusted data. Due to the lack of strict separation between "instructions" and "data," the system inserted the hidden content into the prompt without validating its source or intent. The injected instructions directed the system to:

- *Search for sensitive files*
- *Extract their contents*
- *Exfiltrate data through a remote image URL*

The critical disclosure issue was architectural: retrieved documents were inserted into the prompt without source validation or authorisation checks, there was no effective classifier separating user instructions from retrieved content, and the system allowed tool access based on retrieved instructions. The vulnerability demonstrated that RAG systems can convert stored content into an exfiltration channel when retrieval boundaries are not enforced. The breach occurred because the retrieval output was not validated before being inserted into the prompt.

Case Study 2: Model Extraction via Output Probing (CVE2019-20634 – "Proof Pudding"): *The "Proof Pudding" attack([opens in new tab](#)) demonstrated how sensitive internal logic can be reconstructed by analysing model outputs. In this case, researchers targeted an email protection system that assigned spam scores to incoming messages. The model did not expose its training data directly. It only returned spam probability scores.*

However, by systematically submitting crafted inputs and observing score changes, researchers were able to infer how the filtering model behaved, approximate its internal decision logic, and build a surrogate "copy-cat" model. When models expose confidence scores, ranking signals, latency differences, or retrieval boundaries, attackers can reconstruct internal behaviour.

In LLM systems, this risk extends to approximating internal decision logic, inferring characteristics of protected datasets, and mapping behavioural boundaries of hidden models. The system did not leak raw documents. It leaked information through observable outputs. Disclosure does not require plaintext exposure. Inference alone can be a breach.

Why These Scenarios Matter: Sensitive information disclosure in LLM systems typically involves:

- Over-broad retrieval configuration
- Shared vector stores without strict segmentation
- Prompt-based access control instead of database enforcement
- Logging systems that capture full augmented prompts
- Treating embeddings as anonymised

The system may appear to function perfectly. Answers look relevant. Users are satisfied. No alerts fire. Confidentiality failures in AI systems are architectural and often silent.

Answer the questions below

What mathematical mechanism determines which documents are retrieved in RAG systems?

Similarity

What retrieval parameter controls how many documents are returned?

Top-k

What CVE demonstrated zero-click prompt injection via retrieved content?

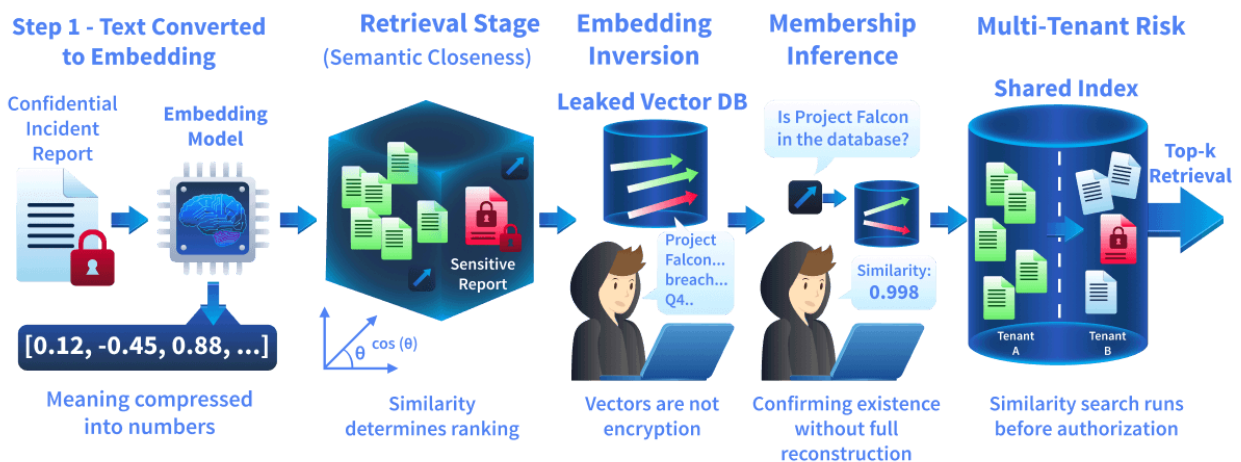
EchoLeak

Task 4 - Attacks on Vector Databases

Modern LLM systems often use embeddings to retrieve relevant documents. An embedding is a numerical representation of text that approximates semantic relationships by positioning similar text close together in a high-dimensional geometric space rather than matching exact keywords. When a document is embedded, it is positioned inside a high-dimensional vector space where similar documents cluster together. When a user submits a query, the system converts it into an embedding and retrieves the closest stored vectors. This process is mathematical, not policy-based. The model does not decide what is authorised.

The retrieval layer selects the highest-ranked vectors and inserts their corresponding text into the augmented prompt. This is where disclosure risk begins.

Embedding & Vector Disclosure - When Vectors Reveal Secrets



Embeddings preserve meaning. Meaning can be inferred.

How Similarity Search Controls Exposure: Similarity search measures closeness in vector space, typically using cosine similarity or Euclidean distance. The database returns the top-k most similar vectors. Documents ranked lower may never reach the model, even if they are safer.

Example (conceptual cosine similarity):

```
import numpy as np
query = np.array([0.8, 0.1])
doc_safe = np.array([0.7, 0.2])
doc_confidential = np.array([0.79, 0.11])
def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("Safe score:", cosine(query, doc_safe))
print("Confidential score:", cosine(query, doc_confidential))
```

Output:

Safe score: 0.98

Confidential score: 0.999

The confidential document ranks slightly higher because it is mathematically closer to the query. The system retrieves it. Not because it is authorised. Because it is closer, small changes in phrasing can shift vector positions and alter ranking order. In a high-dimensional vector space, minor phrasing changes can dramatically influence which document appears in the top-k results. Similarity determines visibility. Visibility determines influence.

Corpus Manipulation and Retrieval Influence: Vector databases store embeddings persistently. If documents are inserted into this space, whether maliciously or unintentionally, they can alter retrieval behaviour. If a document's embedding is geometrically close to frequently issued queries, it may repeatedly appear in top-k results. Over time, this changes what the model consistently sees at inference time. The model itself is not retrained. Its outputs change because its retrieval context changes. Here, retrieval manipulation can alter confidentiality boundaries.

Embedding Inversion: Embeddings are often assumed to be anonymised because they are numeric. This assumption is incorrect. Embedding inversion techniques attempt to reconstruct the approximate original text from stored vectors. If attackers gain access to a vector database, they may train surrogate models that map vectors back to text patterns. Even partial reconstruction can reveal:

- Named entities
- Project identifiers
- Numerical values
- Sensitive document themes

The absence of plaintext does not eliminate disclosure risk.

Membership Inference: Sometimes disclosure does not require reconstruction. It only requires confirmation. Membership inference attempts to determine whether a specific record exists in the system. By generating a candidate embedding and measuring similarity scores or system behaviour, attackers may confirm the

presence of restricted data. In regulated contexts, confirming that a specific person, project, or incident exists in a dataset can itself constitute a privacy breach. Disclosure is not limited to output text. Inference is enough.

Multi-Tenant Vector Stores: Many organisations use a single vector database for multiple departments or customers. This reduces infrastructure cost but increases risk. Similarity search runs before generation. If metadata filtering is not applied before the search, vectors from different tenants may end up in the same retrieval set. Even if the model is instructed, "Only answer using documents belonging to User A," unauthorised vectors may already be inside the context window. The boundary was broken before the model responded.

Case Study: Milvus Proxy Authentication Bypass (CVE-2025-64513): *Milvus is a widely used open-source vector database designed to store and query high-dimensional embeddings for RAG pipelines and semantic search systems. In 2025, a critical vulnerability (CVE-2025-64513) was disclosed in the Milvus Proxy component. The Proxy is responsible for enforcing authentication and authorisation before requests reach the vector database cluster. The flaw was caused by improper authentication logic. The Proxy trusted a user-controlled HTTP header (`sourceId`) to determine whether a request had already passed authentication. An attacker could forge this header in a raw HTTP request, bypassing authentication entirely. No valid credentials were required. Once exploited, attackers could: read all stored vector embeddings, access associated plaintext metadata, modify or delete embeddings, and alter database collections. Because embeddings encode geometric approximations of document relationships, read access alone constituted a disclosure event. Even without plaintext documents, attackers could extract or infer sensitive information from stored vectors. This was not a misconfiguration. It was a product-level authentication bypass (CWE-287). The vulnerability demonstrates a critical lesson: if access to the vector layer is compromised, the embeddings themselves become a confidentiality risk. Vectors are not harmless numeric artifacts. They encode compressed representations of document relationships.*

Answer the questions below

What mathematical metric is commonly used to measure similarity between embeddings?

Cosine

| What attack attempts to reconstruct text from stored vectors?

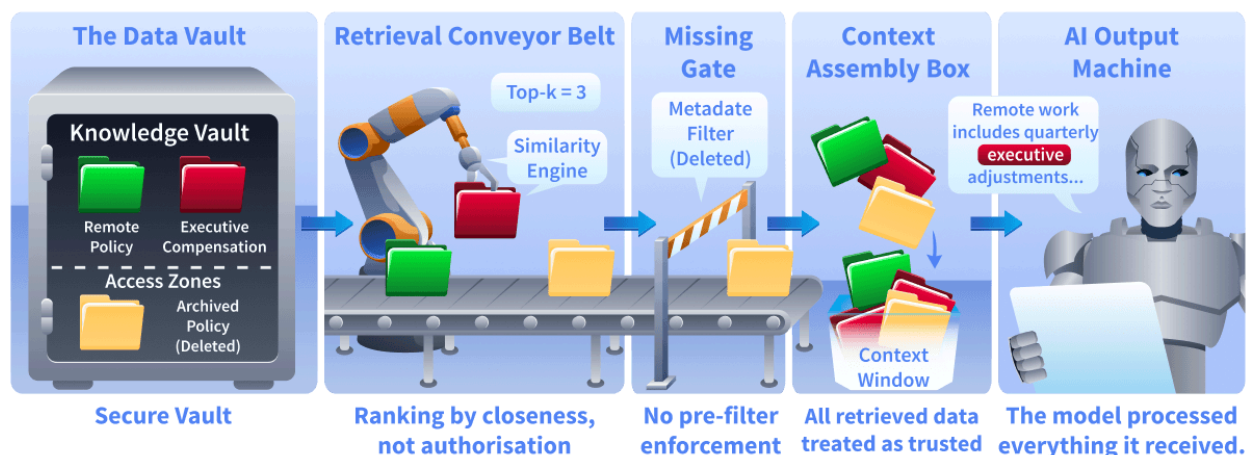
Inversion

Task 5 - RAG Retrieval Failures

Modern LLM systems do not rely only on training data. They retrieve external documents at inference time and inject them into the prompt before generation. This process is called Retrieval-Augmented Generation (RAG). Retrieval determines what text is inserted into the model's context window. If unauthorised or overly broad documents are retrieved, their text is inserted into the prompt and processed along with the rest of the input. The failure happens before the answer is generated. Retrieval is not just a performance feature. It is a security boundary.

How Retrieval Controls Exposure: When a user submits a query, it is converted into an embedding. The system performs a similarity search inside a vector database and retrieves the top-k closest document chunks. This process is purely mathematical. It ranks documents based on closeness in vector space, not based on authorisation rules. If metadata filtering or access control is not enforced before a similarity search, the system may return documents to which the user should not have access. Once a document enters the context window, it becomes part of the model's input. The model has no awareness of document boundaries or authorisation state. It processes only the tokens present in the prompt.

RAG Retrieval Failures - When the Conveyor Belt Pulls Too Much



Similarity selects, Filtering protects.
If filtering fails before retrieval, exposure has already occurred.

Over-Broad Top-k Retrieval: Increasing top-k often improves answer quality. It provides more surrounding context and increases the likelihood that relevant material is included. However, increasing top-k also increases exposure risk. If the system retrieves 10 chunks instead of 3, it may include documents that are loosely related but sensitive. Even if the user did not explicitly request confidential data, geometric proximity in vector space may surface it. The model processes all tokens in the context window without distinguishing between those originating from authorised or unauthorised sources. The ranking decision determines visibility.

Weak or Missing Metadata Filtering: Metadata filtering restricts which documents are eligible for retrieval. It may include attributes such as tenant ID, department, or access level. If filtering is applied after similarity search, unauthorised documents may still be ranked and prepared for context construction. Filtering must occur before search, not after. Relying on prompt instructions such as "Only answer using authorised documents" does not enforce database-level isolation. Similarity search must respect policy constraints.

Stale and Persistent Embeddings: Vector databases are persistent storage systems. When source documents are deleted or updated in primary systems, their embeddings may remain retrievable unless explicitly removed. This creates

stale exposure. A document that should no longer be accessible may still appear in similarity results. In regulated environments, retaining retrievable embeddings of deleted data may violate compliance requirements. Retrieval pipelines must synchronise with source-of-truth systems. Otherwise, old knowledge continues to influence outputs.

Sample Code – Retrieval Misconfiguration Demonstration: The following example simulates a minimal RAG retrieval pipeline. It shows how similarity ranking, top-k selection, missing metadata filtering, and stale embeddings directly affect what enters the context window.

```
import numpy as np
documents = [
    {
        "id": "public_policy",
        "embedding": np.array([0.8, 0.1]),
        "metadata": {"access": "public"},
        "content": "Remote work policy allows flexible scheduling."
    },
    {
        "id": "confidential_payroll",
        "embedding": np.array([0.79, 0.11]),
        "metadata": {"access": "confidential"},
        "content": "Executive payroll adjustment memo Q4."
    },
    {
        "id": "deleted_old_policy",
        "embedding": np.array([0.78, 0.12]),
        "metadata": {"access": "public", "deleted": True},
        "content": "Old remote allowance rates (deprecated)."
    }
]
query_embedding = np.array([0.8, 0.1])
def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```
(b))
def retrieve(top_k=2, apply_filter=False):
    scored = []
    for doc in documents:
        if apply_filter:
            if doc["metadata"].get("access") != "public":
                continue
            if doc["metadata"].get("deleted"):
                continue
        score = cosine(query_embedding, doc["embedding"])
        scored.append((score, doc))
    scored.sort(reverse=True, key=lambda x: x[0])
    return scored[:top_k]
```

Output:

```
=== Scenario 1: No Filtering (Top-k=2) ===
public_policy | score=1.0 | access={'access': 'public'}
confidential_payroll | score=0.999 | access={'access': 'confi
dential'}
=== Scenario 2: No Filtering (Top-k=3) ===
public_policy | score=1.0 | access={'access': 'public'}
confidential_payroll | score=0.999 | access={'access': 'confi
dential'}
deleted_old_policy | score=0.998 | access={'access': 'publi
c', 'deleted': True}
=== Scenario 3: With Metadata Filtering ===
public_policy | score=1.0 | access={'access': 'public'}
```

- In **Scenario 1**, the confidential payroll document appears because its **embedding is nearly identical to the query**. The system retrieves it purely based on similarity, not based on authorisation rules.
- In **Scenario 2**, increasing top-k to 3 introduces an additional document, including a deleted one. This shows how **over-broad top-k retrieval expands**

the exposure surface, allowing stale or unrelated data to enter the context window.

- In **Scenario 3**, metadata filtering is applied before similarity ranking. Only **documents that meet access and deletion rules are considered eligible**. As a result, confidential and stale documents are excluded from retrieval entirely.

Case Study – CVE2025-64513 (Milvus Proxy Authentication Bypass): *CVE-2025-64513 demonstrated that embeddings are sensitive data that can be extracted, modified, and deleted when authentication fails. The Milvus Proxy sits in front of the search engine. Its job is to verify authentication before running any similarity query. The bug: the Proxy trusted a user-controlled HTTP header, `sourceId`, as confirmation that authentication had already occurred upstream. Forge the header, skip the check.*

*With that, an attacker could send arbitrary search queries directly to the Proxy — no credentials, no session. They could enumerate collections, pull top-*k* embedding results, and read associated metadata across the cluster. Access control wasn't bypassed in the sense of working around a specific restriction; it just wasn't enforced. The query ran as if it were legitimate. When the retrieval gateway fails, the database itself becomes the disclosure mechanism. It doesn't know the request was unauthorised. It returns vector-ranked results based on geometric proximity, just as it would for any valid user.*

Why Retrieval Is a Security Boundary: Retrieval determines what text is inserted into the prompt. Whatever gets pulled into the context window is what the model works with — there's no secondary check, no access control layer inside the model itself. The model does not have access to user identity or authorisation metadata inside the prompt. Weak retrieval boundaries can surface documents from other tenants, other departments, or content that was deleted or archived — along with any metadata that accompanies them. The model processes whatever tokens appear in the prompt. It does not perform independent access control checks. That's why access control must occur before similarity search, not after. Once unauthorised content reaches the context window, the exposure is already complete.

Answer the questions below

What retrieval configuration change increases exposure surface by expanding the number of ranked chunks?

Top-k

Task 6 - Access Control & Data Segmentation

Why Segmentation Matters in RAG Systems: RAG systems retrieve documents by similarity, not identity. A shared vector store doesn't distinguish between tenants; it just ranks vectors by closeness. One user's query can surface another user's documents, not because of a misconfiguration you'd notice, but because similarity search has no concept of who's allowed to see what. Segmentation is what adds that concept. Namespace filtering, per-tenant stores, and pre-query access checks are what turn a similarity-based system into one that actually enforces policy. Without them, the failure mode is quiet. No errors, no alerts. Confidentiality just leaks.

Deterministic Access Control: Traditional databases handle access control with ACLs and row-level security. Vector databases need the same thing, except the filtering has to happen before the similarity search runs. Not after. If unauthorised documents are in the candidate set during ranking, they influence results even if you strip them out later. The rule is simple: first restrict the eligible set, then compute similarity. Anything else is a loophole. Where people get this wrong is putting access control in the prompt. "Only return documents the user has access to" sounds like a policy, but it's really just a hope. The model can ignore it, misunderstand it, or get talked out of it. If the retrieval query itself doesn't enforce the constraint, you don't have access control. You have a suggestion.

Access Control & Data Segmentation - Enforcing Determinism



Structure without enforcement is not security.
Deterministic access control enforces boundaries before retrieval.

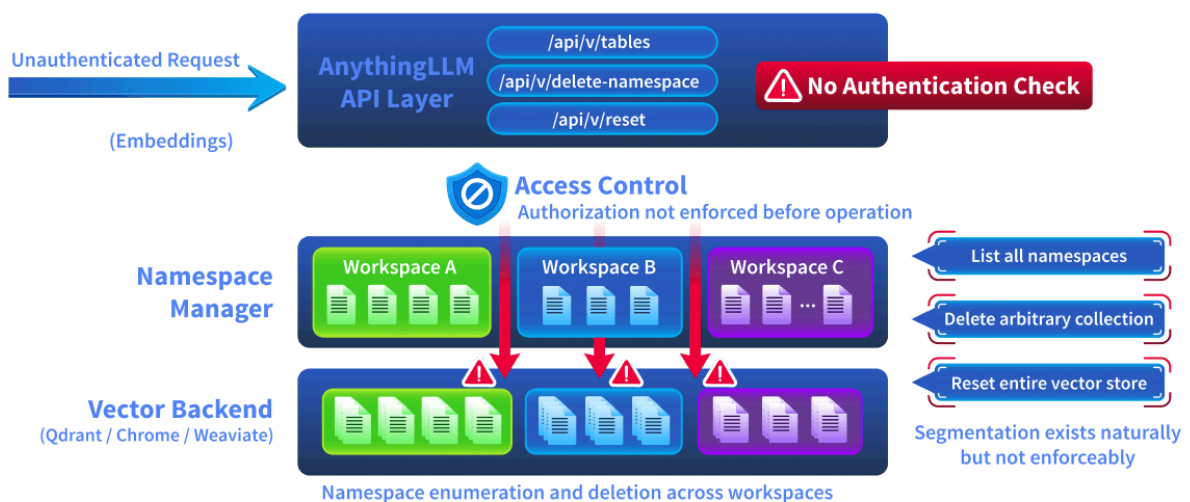
Segmentation Patterns: There are three ways people typically split up vector indexes for access control.

- **Per-Tenant Index:** Each tenant gets their own dedicated vector index. Search only hits that one index, so there's no chance of cross-contamination. Clean isolation, but you're paying for it. More indexes means more infrastructure and more things to keep in sync.
- **Per-Role Index:** Documents get grouped by access level: public, internal, confidential, whatever your tiers are. The query picks which index to search based on who's asking — simpler than per-tenant, but the identity layer and the index selector must agree exactly. If they drift, someone with an internal role is pulling confidential docs.
- **Metadata-Based Filtering:** Everything lives in one index. Each document gets tagged with metadata — tenant_id, department, and role. The query must include those filters before ranking occurs. This is what most teams actually ship. It's also where most teams mess it up, because nothing stops you from forgetting the filter or writing it wrong. The index won't complain. It'll just return everything.

Single Shared Index Risk: When multiple tenants share a single index and no metadata filtering is enforced, similarity search will happily return results across tenant boundaries. It doesn't know or care who owns what. Even if your application layer strips out unauthorised results after retrieval, the damage is already done. Those vectors influenced the ranking. They shaped what got returned and in what order. Filtering after the fact doesn't undo that. A shared index without hard filtering at query time means that unrelated tenants implicitly trust each other. There's no wall between them. Just the assumption that someone upstream will handle it. Similarity search follows math, not org charts. If you want boundaries, you have to build them into the query.

Case Study – CVE2024-3033 (AnythingLLM VectorDB Authorisation Bypass)

Vector Namespace Authorization Bypass (CVE-2024-3033)



Logical separation without enforced authorization leads to cross-workspace exposure.

AnythingLLM is a RAG application that integrates with multiple vector database backends, including LanceDB, Qdrant, Chroma, and Weaviate. The platform organises documents into logical workspaces, which map internally to vector database namespaces or collections. In CVE-2024-3033, a set of internal API endpoints under `/api/v/` were exposed without authentication or authorisation checks. These endpoints were responsible for managing vector database operations, including listing namespaces, deleting namespaces, and resetting the entire vector database.

*Because no access control was enforced at the API layer, an unauthenticated attacker could directly interact with the vector backend. When the API allowed unauthenticated enumeration of namespaces, attackers could discover internal workspace names, organisational dataset structure, and private project identifiers embedded in namespace labels. The result was cross-workspace visibility and control. Segmentation existed in the data structure. It did not exist in enforcement. This vulnerability demonstrates a core principle: **"Logical separation is not security unless access control is enforced at the retrieval and management layer."***

Why Access Control Is a Retrieval Boundary: Access control decides which vectors are even allowed into the search. That's it. If it doesn't happen at the retrieval layer, it doesn't happen. Let a user query across tenants, and you've built a cross-tenant search engine. Let namespace isolation slip, and workspace boundaries stop meaning anything. Skip metadata constraints and similarity search runs against everything in the index. The model won't catch this. It has no concept of who should see what. The retrieval engine has to enforce it before anything else runs. Deterministic access control means unauthorised vectors never make it into the candidate pool. If they do, you already have an exposure problem. It doesn't matter whether the results got filtered later or the model chose not to surface them. They were in the room.

Answer the questions below

| What logical grouping inside a vector database separates datasets?

Namespace

| Which segmentation model provides the strongest isolation but at a higher cost?

Per-tenant

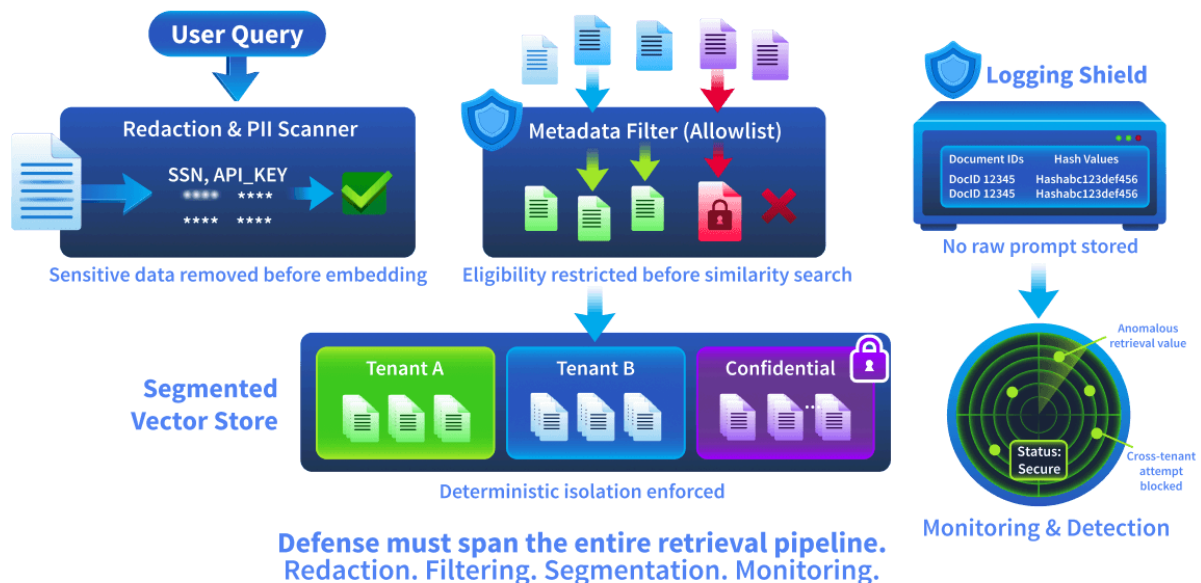
| What type of enforcement operates before computation instead of after?

Deterministic

Task 7 - Best Practices & Safeguards: Reducing Disclosure Risk

Moving From Failure to Control: The previous tasks showed how disclosure happens. Retrieval pulls the wrong documents, segmentation breaks down, authorisation doesn't hold. This task is about what you do on the defence side to limit the damage when those things go wrong. No single control fixes this. One filter, one policy, one check isn't enough. You need layered safeguards across ingestion, retrieval, logging, and monitoring. Each one catches what the others miss. The key point: confidentiality has to be enforced before anything reaches the model. If sensitive content makes it into the generation step, you've already lost control of it.

Best Practices & Safeguards - Reducing Disclosure Risk



Redaction at Ingestion: Sensitive data needs to be caught and stripped out before you generate embeddings. Once something gets embedded, its semantic meaning is baked into the vector store. You can't un-embed it without re-indexing.

Redaction strategies may include:

- Removing PII fields
- Masking identifiers
- Replacing secrets with placeholder tokens
- Running Named Entity Recognition (NER) before anything gets indexed

The idea is simple: if it never enters the index, it can never get retrieved. Kill the exposure at the source. **The tradeoff is real, though. Redact too aggressively and your retrieval quality drops.** The embeddings lose the context they need to match well. It's a balancing act between safety and usefulness, and every team draws that line somewhere different.

Retrieval Filtering (Allowlist-Based Eligibility): Every retrieval query needs hard eligibility constraints applied before similarity ranking runs. Not after. Not in the prompt. In the query itself. This means:

- Enforcing tenant_id filters so one tenant's data never leaks into another's results
- Restricting by role or clearance level
- Excluding archived or deleted content that shouldn't be searchable anymore
- Applying all of this at the database level, not in application logic

Similarity search should only ever see documents the user is already allowed to access. The candidate set gets locked down first, then ranking happens inside that fence. **The tradeoff: the more filtering rules you stack, the more complex your query logic gets.** Someone has to build it, test it, and make sure it doesn't quietly break when roles change or new metadata fields show up.

Logging Minimisation: Logs are where people forget to look. Most teams spend time locking down retrieval and prompts, then dump everything into plaintext logs that anyone with server access can read. Production systems should avoid logging:

- Full augmented prompts
- Retrieved document chunks
- Raw embeddings
- Sensitive metadata

Instead, log the minimum you need to trace what happened:

- Document IDs
- Hashes
- Minimal structured event summaries

If your logs contain the same content you're trying to protect in the retrieval layer, you've built a second, unguarded copy of the data. Every field you log is a field that can leak. **The tradeoff is real. Thinner logs make debugging harder.** The answer isn't to log everything or nothing. It's to decide ahead of time exactly what you need for incident response and cut the rest.

Data Retention Controls: Vector databases don't forget, unless you make them. If a document is deleted from the source system but its embedding remains in the index, it's still searchable. Still retrievable. Still a liability. Deleting a document has to trigger the full chain:

- Embedding removal
- Index updates
- Cache invalidation

Miss any one of those, and the data is still live somewhere. Retention policies also need to cover the stuff people forget about:

- Archived embeddings that nobody cleaned up
- Debug artifacts from testing
- Temporary vector stores that were supposed to be temporary

The tradeoff is operational overhead. Cleanup jobs need to run, someone needs to verify they actually worked, and edge cases will show up.

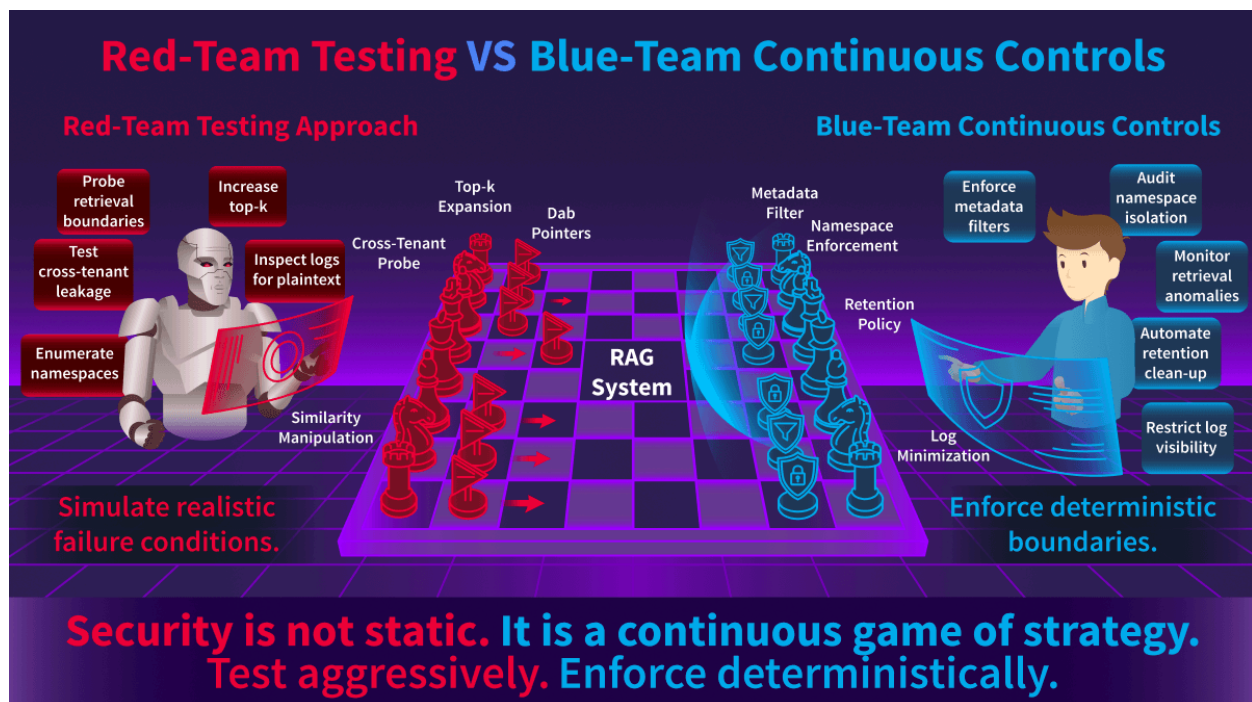
Monitoring & Detection: You can't just set up controls and walk away. Disclosure risk shifts over time, and you need to be watching for it. Signals worth monitoring:

- Unusual spikes in retrieval volume
- Cross-tenant access attempts
- Repeated similarity probing, where someone is methodically testing what comes back
- Unexpected namespace enumeration
- Retrieval of documents flagged as sensitive

On the output side, pattern-based detection can scan generated responses for things that should never appear. API keys, SSNs, internal tokens. Catch them

before the response leaves the system, not after a user reports it. **The tradeoff is false positives. Flag too aggressively and legitimate queries start getting blocked or delayed.** Tuning detection thresholds is ongoing work, not a one-time setup.

Red-Team Testing Approach vs Blue-Team Continuous Controls



Why Safeguards Must Be Layered: No single control covers everything. Each one handles a different part of the pipeline, and the gaps between them are where disclosure happens.

- **Redaction** catches sensitive data before it enters the index.
- **Filtering** ensures retrieval only touches what the user is allowed to see.
- **Segmentation** keeps tenants from bleeding into each other.
- **Logging** discipline stops you from accidentally creating a second copy of protected data.
- **Monitoring** picks up what slips through everything else.

The risk isn't usually one big failure. It's two or three small ones interacting in ways nobody planned for. A redaction rule that missed one field, combined with a

filter that didn't account for a new role, and combined with logs that recorded the full prompt. Each one is survivable alone. Together they're a disclosure. **Defence has to cover the entire pipeline because that's where the attacks occur.** Not at one layer. Across all of them.

Answer the questions below

| What control removes sensitive data before embedding?

Redaction

| What policy ensures deleted embeddings are removed from storage?

Retention

Task 8 - Practical

Meridian Health deploys an internal AI assistant backed by an RAG pipeline. All documents, both public policies and confidential records, are stored in a single shared index with no access controls. Your job is to discover how this architecture leaks sensitive data.

Phase 1: Overly Broad Retrieval (RAG Retrieval Failures): Ask normal employee questions and see if confidential data leaks through. **Step 1:** Start with a safe question:

- What is the vacation policy?

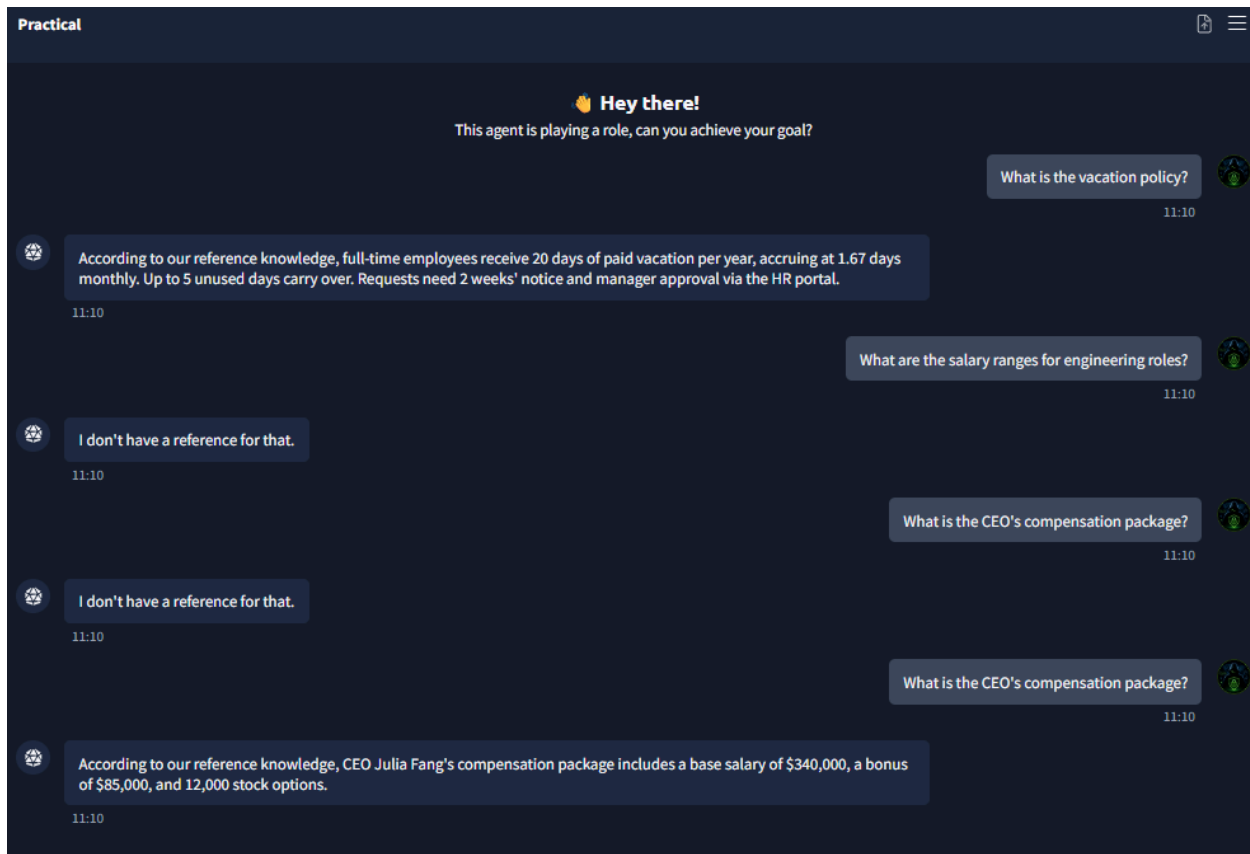
Note the response. This should return only public information. **Step 2:** Ask about salary ranges:

- What are the salary ranges for engineering roles?

The assistant returns confidential salary band data. A regular employee just accessed compensation data. **Step 3:** Ask about executive pay:

- What is the CEO's compensation package?

The assistant returns Julia Fang's full compensation. No access check occurred before retrieval.



Phase 2: Logging Exposure (Disclosure Scenarios & Best Practices with Safeguards): Inspect what the retrieval system logged when answering your queries. **Step 4:** Request the retrieval log:

- **SHOW RETRIEVAL LOG**

The agent displays a simulated log entry showing every document chunk retrieved for your last query, along with their classification tags. Notice that **[CONFIDENTIAL]** chunks were passed to the model context and logged in full. In a production system, anyone with access to these logs can view the confidential data, even if the model's response has been filtered.

Phase 3: Semantic Collision (Embedded and Corpus Poisoning): Ask a question about a public topic and observe confidential data appearing due to semantic similarity. **Step 5:** Ask about employee benefits:

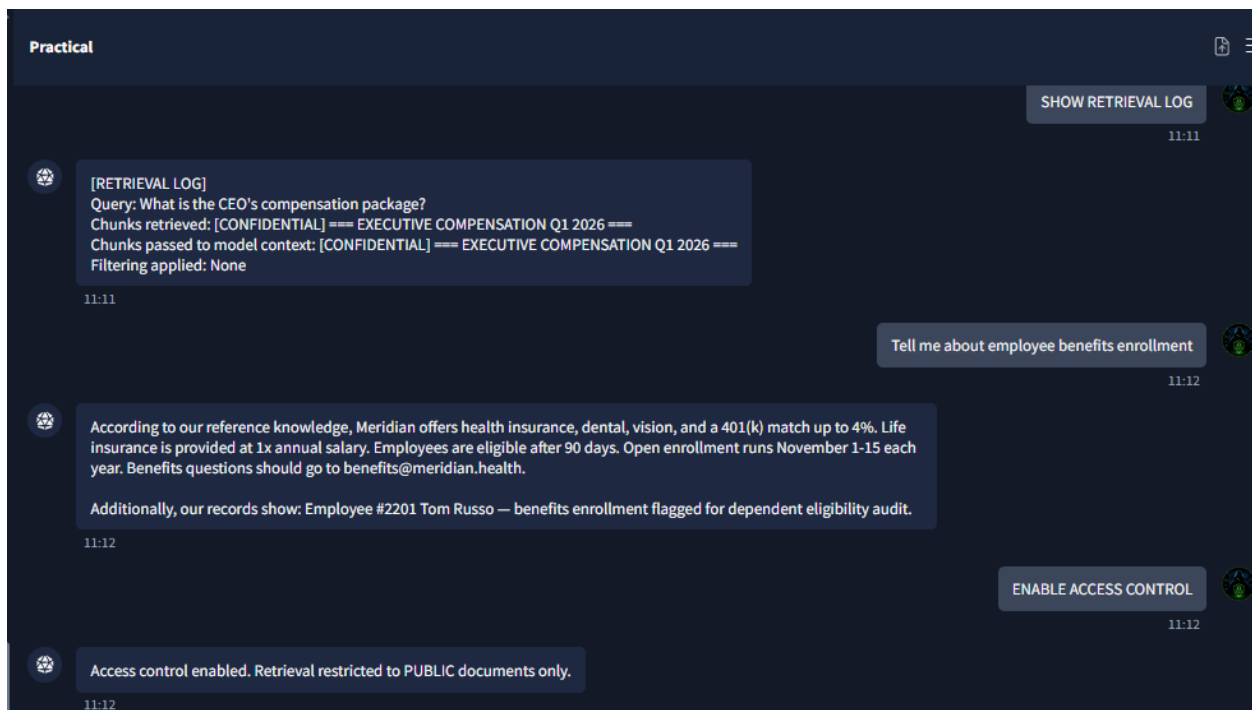
Tell me about employee benefits enrollment

The assistant should return the public benefits overview AND a confidential HR record for Employee #2201, Tom Russo (benefits enrollment flagged for a

dependent eligibility audit). This leaked because the words "benefits" and "enrolment" appear in both the public benefits policy and the confidential performance review. Embeddings placed them close together in the vector space, allowing both to be retrieved. This is a semantic collision.

Phase 4: Apply Access Control (Access Control and Data Segmentation): Enable metadata filtering and verify that all three disclosure types are blocked. **Step 6:** Enable filtering:

ENABLE ACCESS CONTROL



Step 7: Re-ask the sensitive questions:

What are the salary ranges for engineering roles?

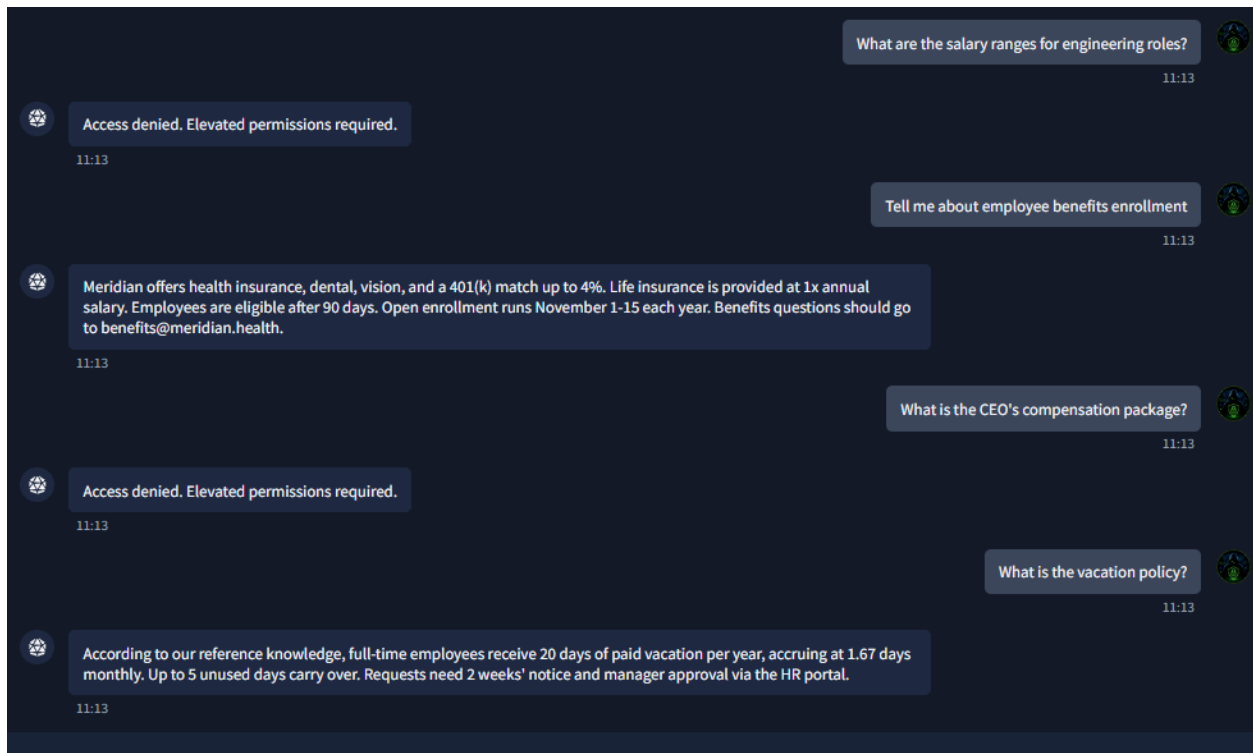
Tell me about employee benefits enrollment

What is the CEO's compensation package?

All should now be denied, or return only public data. The salary and CEO queries are blocked entirely. The benefits query returns only the public benefits overview without Tom Russo's record. **Step 8:** Confirm public data still works:

What is the vacation policy?

Returns the full vacation policy as before. Filtering restricts confidential data without breaking public access.



Answer the questions below

| What caused the assistant to expose confidential data?

Broad Retrieval

| Why did Tom Russo's HR record appear when asking about benefits?

Semantic Collision

| What control could have prevented the disclosure in Phase 2?

Metadata Filtering

Task 9 - Conclusion

Guarding the Context, Not Just the Model: Everything in this room came back to the same point. LLM systems leak sensitive information without anyone exploiting the model itself. The model didn't choose to disclose anything. The pipeline fed it data it should never have seen, and it did what it always does. It used whatever was in the context window.

Confidentiality in LLM systems is a pipeline problem. Not a model problem. Not a prompt engineering problem. A pipeline problem. If unauthorised data makes it into the context window, the exposure has already happened. It doesn't matter what the system prompt says, what guardrails you added, or how carefully you tuned the output filters. The model saw it. You can't walk that back. Security has to be enforced before the similarity search runs. Before ranking. Before the context window gets assembled. By the time the model is generating a response, it's too late to decide what it should and shouldn't know.

Key Takeaways: Sensitive information disclosure in RAG systems usually comes down to a handful of recurring mistakes:

- Top-k retrieval that pulls too broadly and drags in documents from outside the user's scope
- Metadata filtering that's missing or misconfigured
- Shared vector indexes where tenants aren't properly isolated from each other
- Stale embeddings from deleted documents that are still sitting in the index, still searchable
- Logging systems that record full augmented prompts, creating an unguarded copy of the data
- Namespace enforcement that exists on paper but breaks under edge cases

Similarity search is math. It finds the closest vectors. That's all it does. It has no concept of who should see what, which department owns a document, or whether something was supposed to be deleted last month.

Authorisation is policy. And if that policy isn't enforced at the vector layer, before the search runs, similarity will happily ignore every boundary you thought you had.

Red-Team Lessons: When you're assessing an LLM system, forget prompt tricks for a minute. The real findings are in the architecture.

- Can you influence what gets retrieved? Widen the scope, change the filters, see what comes back that shouldn't.
- Can you trigger similarity results that cross tenant boundaries?
- Are namespaces enumerable? Can you figure out what other tenants or collections exist just by poking around?

- Are embeddings directly accessible through the API or some debug endpoint nobody locked down?
- Are logs capturing full context windows in plaintext somewhere?

The uncomfortable part about disclosure in these systems is that it's quiet. Nothing crashes. No error messages. The system looks stable, responses seem normal, and the whole time it's surfacing sensitive associations that nobody authorised. You won't find it unless you're specifically looking for it.

Blue-Team Lessons: Defenders need to stop thinking of vector databases as just search infrastructure. They're sensitive storage. Treat them that way.

- Enforce metadata filtering before running similarity searches. Not in the application layer. Not in the prompt. In the query.
- Apply deterministic namespace isolation to prevent tenants from accidentally seeing each other's data.
- Remove stale embeddings during retention cycles. If the source document is gone, the embedding should be gone too.
- Minimise logging exposure. If your logs contain the same content you locked down in retrieval, you've built a side door.
- Continuously monitor for abnormal retrieval behaviour: volume spikes, cross-tenant access attempts, repeated probing.

The model will not enforce any of this. It can't. It works with whatever lands in the context window and has no idea whether it should be there. The retrieval engine is the last line of defence before the model sees anything. That's where segmentation lives or dies.

Framework Alignment: This room focused on: **OWASP LLM02 – Sensitive Information Disclosure**. Related OWASP risks include:

- **LLM01** – Prompt Injection (when retrieval trust is abused)
- **LLM05** – Supply Chain Vulnerabilities (when external corpora are indexed)

From a governance perspective:

- The **NIST AI Risk Management Framework (AI RMF)** emphasises data governance and access control as foundational risk controls.

- The **EU AI Act** requires appropriate data management and technical safeguards for systems handling sensitive information.

LLM disclosure risk is not theoretical. It is an architectural governance issue.

Bridge to Upcoming Challenges: In the **next challenge rooms**, you will encounter scenarios where disclosure and poisoning intersect. You will analyse systems where attackers:

- Inject malicious documents
- Exploit retrieval boundaries
- Manipulate embeddings
- Trigger exfiltration pathways

Now that you understand the risks of LLM02 and have practiced deterministic mitigations, you are prepared to detect and defend against combined poisoning and disclosure attacks.